Poseidon Package Specification v3.0.0

PDF version of the standard at https://github.com/poseidon-framework/poseidon-schema
generated on 2026-03-23 based on the Git commit 3fc74a8

Table of contents

I The Poseidon Standard v3.0.0

Poseidon is a solution for archaeogenetic genotype data organisation. It is geared towards human data, but is to a large extent species-agnostic and can be used to track archaeogenetic data also of non-human species.

This standard defines a data structure: the **Poseidon package**. A Poseidon package stores genotype data with meta- and context information.

A .pdf version of the latest instance of this document can be downloaded here.

Further details on genotype data, the .janno file and the .ssf file are documented on the Poseidon website.

A changelog documents the changes across different schema versions here.

The key words *MUST*, *MUST NOT*, *REQUIRED*, *SHALL*, *SHALL NOT*, *SHOULD*, *SHOULD NOT*, *RECOMMENDED*, *MAY*, and *OPTIONAL* in this document are to be interpreted as described in RFC 2119.

1 Primary entities of a Poseidon package

The main operational entities in a Poseidon package are discrete sets of genotype data attributed to a single human or non-human individual, scientifically generated for archaeogenetic research questions. Within a Poseidon package each of these sets gets attributed a unique identifier: the `Poseidon_ID`.

Generally, archaeogenetics operates on depositional contexts, e.g. graves, with one or multiple (ancient) human or non-human individuals. Usually, it is possible to attribute the (skeletal) remains within these contexts to individuals based on archaeological evidence and physical-anthropological analysis. Each individual can get sampled one or multiple times, either by directly probing their preserved tissue, or by sampling any reagent that contains their DNA (through whatever pathway or taphonomic process). From one such sample one or multiple extracts can be derived, which can be transformed into one or multiple libraries, which may or may not be subjected to a DNA capture protocol and then sequenced one or multiple times. The raw sequencing data can undergo various different forms of computational processing and eventually genotyping to produce the data relevant for most derived analyses and thus stored in a Poseidon package.

While the wetlab-processes yield a relatively predictable tree of separate physical and digital products for any given sample, the computational data-processing breaks the conceptual tree-ness by allowing for arbitrary conflation of sequencing data obtained through potentially separate means: Data from different libraries, for example, may be merged if they are from the same individual, even if they are not from the same sample.

`Poseidon_ID`s therefore represent one consciously selected end-point in the complex data preparation graph laid out above. Typically this end-point corresponds to an optimal result for a given individual, research question and publication.

For the sake of convenience and despite the lack of conceptual clarity, below we sometimes use the term *sample* to denote `Poseidon_ID` entities. Data aggregation on the level of physical samples is often sensible, and the term is conventionally used for analysis endpoints in the community of practice.

2 The Poseidon package structure

A Poseidon package is defined by the POSEIDON.yml file, which holds relative paths to all other files in the package.

A package therefore MUST contain:

- A `POSEIDON.yml` file to formally define the package
- Genotype data in PLINK, EIGENSTRAT or VCF format

It SHOULD additionally contain:

- A `.janno` file to store context information on spatiotemporal origin or sample quality
- A `.bib` file for literature references

It MAY also contain:

- A `README.md` file for arbitrary, additional context information
- A `CHANGELOG.md` file to document changes to the package

- A `.ssf` file with information on the underlying raw sequencing data

Here is an example of a package `Switzerland_LNBA_Roswita` in one directory:

```
Switzerland_LNBA_Roswita/POSEIDON.yml
Switzerland_LNBA_Roswita/Switzerland_LNBA.bed
Switzerland_LNBA_Roswita/Switzerland_LNBA.bim
Switzerland_LNBA_Roswita/Switzerland_LNBA.fam
Switzerland_LNBA_Roswita/Switzerland_LNBA.janno
Switzerland_LNBA_Roswita/Switzerland_LNBA.ssf
Switzerland_LNBA_Roswita/Switzerland_LNBA.bib
Switzerland_LNBA_Roswita/README.md
Switzerland_LNBA_Roswita/CHANGELOG.md
```

3 Text encoding

All text files in the package MUST be UTF-8 encoded. They SHOULD use Unix-style line endings, so a single Line Feed (LF, `\n`) character, NOT a Carriage Return and Line Feed (CRLF) pair (`\r\n`) as in MS DOS and Windows.

`Poseidon_ID`s and `Group_Name`s, so the primary sample and group identifiers across `.janno`, `.ssf`, and genotype data files, MUST contain only characters of a subset of the 7-bit ASCII code set. Specifically the alphanumeric characters `A-Z`, `a-z`, `0-9`, and the symbols `_` (underscore), `-` (hyphen-minus), and `.` (period, dot or full stop).

4 The `POSEIDON.yml` file

The `POSEIDON.yml` file defines Poseidon packages by listing metainformation and relative paths in a standardised, machine-readable format.

- It MUST be a valid YAML file.
- Its mandatory and optional fields are documented in the POSEIDON_yml_fields.tsv file.

Here is an example for a `POSEIDON.yml` file:

```yaml
poseidonVersion: 2.7.1
title: Switzerland_LNBA_Roswita
description: LNBA Switzerland genetic data not yet published
contributor:
  - name: Roswita Malone
    email: roswita.malone@example.org
    orcid: 1234-1234-1234-1234
  - name: Paul Panther
    email: paul.panther@example.edu
packageVersion: 1.1.2
lastModified: 2021-01-28
licence:
  name: CC BY 4.0
  file: LICENCE.md
```

```yaml
genotypeData:
  format: PLINK
  genoFile: Switzerland_LNBA_Roswita.bed
  genoFileChkSum: 95b093eefacc1d6499afcfe89b15d56c
  snpFile: Switzerland_LNBA_Roswita.bim
  snpFileChkSum: 6771d7c873219039ba3d5bdd96031ce3
  indFile: Switzerland_LNBA_Roswita.fam
  indFileChkSum: f77dc756666dbfef3bb35191ae15a167
  snpSet: 1240K
jannoFile : Switzerland_LNBA_Roswita.janno
jannoFileChkSum: 555d7733135ebcabd032d581381c5d6f
sequencingSourceFile: Switzerland_LNBA_Roswita.ssf
sequencingSourceFileChkSum: 19db1906240ee2f076e1a9659567dca4
bibFile: Switzerland_LNBA_Roswita.bib
bibFileChkSum: 70cd3d5801cee8a93fc2eb40a99c63fa
readmeFile: README.md
changelogFile: CHANGELOG.md
```

When a package is modified in any way (including updates of the context information in the `.janno` file), then the `packageVersion` field SHOULD be incremented and the `lastModified` field updated to the current date.

4.1 Package versioning

The `packageVersion` field is a mandatory entry of the `POSEIDON.yml` file. It denotes the version of the individual package, using a three-component versioning system derived from semantic versioning.

Each version number is comprised of three numbers, separated by a `.`. For example: `0.1.0`, `1.0.0` or `2.1.3`. The first number gives the `Major`, the second the `Minor` and the third the `Patch` component of the version number. For a Poseidon package these components SHOULD be incremented when the following changes occur:

- **`Major`** (e.g. `1.4.2` -> `2.0.0`)
  - When samples are added to a package.
  - When samples are removed from a package.
  - When the genotype data (i.e. the contents of the `.bed`/`.bim`/`.fam` or `.geno`/`.snp`/`.ind` files) for any number of samples is changed.
- **`Minor`** (e.g. `1.4.2` -> `1.5.0`)
  - When larger pieces of meta- or context information are added or modified in any package file, except the genotype data. For example:
    - An entire `.janno`, `.bib` or `.ssf` file is added or replaced.
    - Entire columns in the `.janno` or `.ssf` file are added or replaced.
    - Primary publications for samples in the `.janno` and `.bib` file are added or replaced.
- **`Patch`** (e.g. `1.4.2` -> `1.4.3`)
  - When smaller pieces of meta- or context information are added or modified in any package file, except the genotype data. For example:
    - Individual entries in the `.janno` or `.ssf` file are added or replaced.

- ∗ Secondary publications for samples in the `.janno` and `.bib` file are added or replaced.
- ∗ BibTeX entries in the `.bib` file are modified.
- ∗ The package `description` changes in the `POSEIDON.yml` file.
- ∗ The `CHANGELOG.md` file is modified with additional information on previous entries.

When the `packageVersion` is changed, then the `lastModified` date MUST be updated and an entry to the `CHANGELOG.md` file SHOULD be added summarising the changes made.

Packages SHOULD start at `packageVersion` `0.1.0`.

5 Data licensing and the LICENCE.md file

Data licences are a common way to grant the public permission to use a dataset under copyright law.

Poseidon packages MAY specify a licence, and if so, SHOULD use Creative Commons licences.

Licences are documented in the `POSEIDON.yml` file in the `licence` section, either with just the `name`, or with a licence `file`, or with both the `name` and a `file`. `name` SHOULD include a short string with name and version of the licence, e.g. `CC BY 4.0`. The `file`, typically named `LICENCE.md`, MAY include the full text of a licence, or a short notifier further contextualizing the entry in the `name` field. For example:

```
The Poseidon package Switzerland_LNBA_Roswita © 2021 by Roswita Malone is licensed under
↪ Creative Commons Attribution 4.0 International. To view a copy of this license, visit
↪ https://creativecommons.org/licenses/by/4.0/
```

6 Genotype data

Genotype data in Poseidon packages is stored either in (binary) PLINK, EIGENSTRAT or Variant Call Format (VCF).

	PLINK (binary)	EIGENSTRAT	VCF
genotype file	`.bed` (binary biallelic genotype table) or `.bed.gz`	`.geno` (genotype file) or `.geno.gz`	`.vcf` or `.vcf.gz`
SNP file	`.bim` (extended MAP file) or `.bim.gz`	`.snp` (snp file) or `.snp.gz`	
individual file	`.fam` (sample information)	`.ind` (indiv file)	

Both PLINK and EIGENSTRAT formats require three files to be specified. In PLINK, the genotype file is binary (with 2 bits per genotype), while in Eigenstrat, the genotype file is text-based (with 8 bits per genotype). The SNP and individual files are text-based for both formats (see links behind the file endings in the table above). The EIGENSTRAT format specifically is common within archaeogenetics, compatible with many important tools, e.g. EIGENSOFT and ADMIXTOOLS. Finally, the VCF format is the most formally specified format, with properly versioned specifications being released regularly. VCF is well established in the wider genetics community and the de-facto standard to store variants in the field of medical genetics.

VCF files, as well as genotype and SNP files in PLINK and EIGENSTRAT can be stored in gzipped form, signified by an additional file ending (`*.gz`).

To make VCF files fully convertible to PLINK and EIGENSTRAT, they MUST be biallelic and contain only genotypes coded as `0/0`, `0/1`, `1/1`, `./.`. Furthermore, they CAN encode group names and genetic sex for all samples through special header fields `##group_names=name1,name2,...` and `##genetic_sex=F,U,M,...`, respectively. If these fields are not present, then group names are assumed to be "unknown" and genetic sex "U" (unknown) for all samples.

7 The `.janno` file

The `.janno` file is a tab-separated text file with a header line. It holds context information (variables/columns) for each sample (objects/rows) in a package.

- A set of strictly defined core variables (defined by column name) and their possible content are documented here: janno_columns.tsv
- A `.janno` file MAY have all of these core variables, or only a subset of them.
- Only three columns MUST be present to make the file valid: **Poseidon_ID**, **Group_Name** and **Genetic_Sex**.
- Arbitrary columns not defined here MAY be added as long as their column names do not clash with the defined ones.
- Arbitrary, additional free-text information directly related to a column from the set of specified core variables in janno_columns.tsv SHOULD be added in a column whose name has the form **_Note**. Example: `Contamination_Note`.
- The column order is not fixed, but MAY follow the order in janno_columns.tsv. **_Note** columns SHOULD be placed directly after the respective column they are refering to.
- If information is unknown or a variable does not apply for a certain sample, then the respective cell(s) MAY be filled with `n/a` or simply an empty string.
- The order of the samples (rows) in the `.janno` file MUST be equal to the order in the genetic data files (`.ind`, `.fam`) in the package.
- The values in the columns **Poseidon_ID**, **Group_Name** and **Genetic_Sex** MUST be equal to the terms used in the genetic data files (`.ind`, `.fam`).
- Multiple predefined columns of the `.janno` file are list columns that can hold multiple values (either strings or numerics) separated by `;`.
- The decimal separator for all floating point numbers MUST be `.`.

8 The `.bib` file

A BibTeX file with all references listed in the `.janno` file. The entry keys MUST fit the ones used in the `.janno` file.

Example:

```
@article{CassidyPNAS2015,
    doi = {10.1073/pnas.1518445113},
```

```
    url = {https://doi.org/10.1073%2Fpnas.1518445113},
    year = 2015,
    month = {dec},
    publisher = {Proceedings of the National Academy of Sciences},
    volume = {113},
    number = {2},
    pages = {368--373},
    author = {Lara M. Cassidy and Rui Martiniano and Eileen M. Murphy and Matthew D. Teasdale and James Mallory and Barrie Hartwell and Daniel G. Bradley},
    title = {Neolithic and Bronze Age migration to Ireland and establishment of the insular Atlantic genome},
    journal = {Proceedings of the National Academy of Sciences}
}
```

To connect a sample in the package to this particular literature reference, the .janno file column `Publication` would have to be filled with `CassidyPNAS2015`.

9 The `README.md` file

A simple markdown file with informal, arbitrarily structured information accompanying the package.

Example:

```
This package contains a rather interesting set of samples relevant for the peopling of the Territory of Christmas Island in the Indian Ocean. We consider this especially relevant, because ...
```

10 The `CHANGELOG.md` file

A markdown file to document changes in the history of a package.

Example:

```
- V 1.1.1: Fixed a spelling mistake in one site name: "Hosenacker" -> "Rosenacker"
- V 1.1.0: Added mtDNA contamination estimation to the .janno file
- V 1.0.0: Added spatial coordinates and age information to the .janno file and finalized a first stable version of the package
- V 0.2.0: Added previously restricted sample L1337
- V 0.1.0: Creation of the package
```

The structure with `- V X.X.X:` at the beginning of each line is not mandatory, but SHOULD be followed for reasons of interoperability.

11 The .ssf file

The `.ssf` file is another tab-separated text file with a header line. It stores sequencing source data, so metainformation about the raw sequencing data behind the genotypes in a Poseidon package. The primary entities in this table are sequencing entities, typically corresponding to DNA libraries or even multiple runs/lanes of the same library.

- The predefined columns are specified here: ssf_columns.tsv
- All columns of this schema are optional, so a `.ssf` MAY have all of these core variables, only a subset of them, or even none. It SHOULD have a `poseidon_IDs` column, though, to link the sequencing entities to the Poseidon package.
- The link to the individuals listed in the `.janno`-file (and therefore to the entire Poseidon package) is made through a many-to-many foreign-key relationship between the .janno column `Poseidon_ID` and the .ssf column `poseidon_IDs`. That means each entry in the .janno file can be linked to many rows in the .ssf file and vice versa.
- As in the `.janno` file arbitrary columns not defined here MAY be added to the `.ssf` file as long as their column names do not clash with the defined ones.
- The order of columns and rows is irrelevant.
- If information is unknown or a variable does not apply, then the respective cell(s) MAY be filled with `n/a` or simply an empty string.
- Multiple predefined columns of the `.ssf` file are list columns that can hold multiple values (either strings or numerics) separated by `;`.
- The decimal separator for all floating point numbers MUST be `.`.

12 Details

12.1 The `Capture_Type` .janno column

The following protocols are specified:

- `Shotgun`: Sequencing without any enrichment (whole genome sequencing, screening etc.).
- `1240K`: Target enrichment with hybridization capture optimised for sequences covering the 1240k SNP array, see @Fu2015, @Haak2015, @Mathieson2015.
- `ArborComplete`, `ArborPrimePlus`, `ArborAncestralPlus`: Target enrichment with hybridization capture as provided by Arbor Biosciences in three different kits branded myBaits Expert Human Affinities.
- `TwistAncientDNA`: Target enrichment with hybridization capture as provided by Twist Bioscience @Rohland2022.
- `WISC2013`: Whole genome capture as described by @Carpenter2013.
- `OtherCapture`: Target enrichment with hybridization capture for any other set of sequences.

II Appendix

The following tables specify individual fields/variables/columns in the `POSEIDON.yml`, the `.janno` and the `.ssf file`.

An asterisk `*` after the field name indicates a mandatory field that a given file MUST include to be valid.

1 POSEIDON.yml file fields

POSEIDON.yml file fields

Field	Description
poseidonVersion*	Poseidon package format version (e.g. 2.0.1) type: String format: X.Y.Z
title*	title of the package type: String
description	description of the package (one or multiple sentences) type: String
contributor	list of contributors to the package (not the publication author, but the Poseidon package creator) type: Array
name*	name of one contributor subfield of: contributor type: String
email*	email of one contributor subfield of: contributor type: String format: Email
orcid	orcid of one contributor subfield of: contributor type: String format: ORCID
packageVersion*	package version (should be changed/incremented when the package is changed) type: String format: X.Y.Z

POSEIDON.yml file fields *(continued)*

Field	Description
lastModified	date of last modification of the package (should be updated when the package is changed) type: Date format: YYYY-MM-DD
licence	data licence section
name	short name of data licence that applies for this package, usually a Creative Commons licence subfield of: licence type: String
file	relative path to a licence file (usually not necessary, the name is sufficient for standard licences) subfield of: licence type: String format: Path
genotypeData*	genotype data section
referenceGenomeAssembly	reference genome name of the reference genome used, e.g. GRCh37 subfield of: genotypeData type: String
referenceGenomeAssemblyURL	reference assembly accession URL from a public database, such as NCBI or Ensembl subfield of: genotypeData type: String format: URL
format*	genotype data file format subfield of: genotypeData type: String format: EIGENSTRAT;PLINK;VCF
genoFile*	relative path to the genotype file. If gzipped, MUST end with *.gz subfield of: genotypeData type: String format: Path

POSEIDON.yml file fields *(continued)*

Field	Description
genoFileChkSum	md5 checksum of the genotype file subfield of: genotypeData type: String format: md5 hash
snpFile*	relative path to the snp file. If gzipped, MUST end with *.gz subfield of: genotypeData type: String format: Path
snpFileChkSum	md5 checksum of the snp file subfield of: genotypeData type: String format: md5 hash
indFile*	relative path to the ind file subfield of: genotypeData type: String format: Path
indFileChkSum	md5 checksum of the ind file subfield of: genotypeData type: String format: md5 hash
snpSet	SNP set in the genotype data subfield of: genotypeData type: String format: 1240K;HumanOrigins;Other
jannoFile	relative path to the .janno file type: String format: Path
jannoFileChkSum	md5 checksum of the .janno file type: String format: md5 hash
sequencingSourceFile	relative path to the .ssf file type: String format: Path

POSEIDON.yml file fields *(continued)*

Field	Description
sequencingSourceFileChkSum	md5 checksum of the .ssf file type: String format: md5 hash
bibFile	relative path to the .bib file type: String format: Path
bibFileChkSum	md5 checksum of the .bib file type: String format: md5 hash
readmeFile	relative path to the README file type: String format: Path
changelogFile	relative path to the CHANGELOG file type: String format: Path

2 .janno file variables

.janno file variables

Variable	Description
Poseidon_ID*	sample identifier as defined by the genetics laboratory (e.g. I1234, BOT001), must contain only the ASCII characters "A-Za-z0-9_-.", must fit to the values in the Poseidon package .fam/.ind file, must be unique within one package, if multiple datasets exist for the same individual different Poseidon_IDs are required type: String
Genetic_Sex*	genetic sex of the individual derived from this sample, only F, M or U because the EIGENSTRAT and PLINK formats only support these three, edge cases (e.g. XXY, XYY, X0) can be documented in Chromosomal_Anomalies type: Char allowed values: F; M; U

.janno file variables *(continued)*

Variable	Description
Group_Name*	meaningful population/group identifiers for the sample, must contain only the ASCII characters "A-Za-z0-9_-.", can follow the geographic-temporal nomenclature proposed by Eisenmann et al. 2018 (https://doi.org/10.1038/s41598-018-31123-z), or communicate additional categories that are meaningful for groupings in specific analyses, such as cultural labels, outlier status or relatedness to other samples, multiple entries separated by ;, the first value must be equal the group name in the .fam/.ind file list column type: String
Individual_ID	identifier for the sampled individual type: String
Species	Species name of the sample. Should follow binomial nomenclature as standard in Biology, e.g. Homo sapiens. type: String
Alternative_IDs	alternative identifiers for the same sampled individual, e.g. IDs in other databases or popular names like Ötzi/Iceman. Alternative_IDs and Alternative_IDs_Context must have the same number and order of entries list column type: String
Alternative_IDs_Context	context of Alternative_IDs, so e.g. the name of the database (like AADRv62) where a respective identifier is used, or just "popular" for a common non-scientific name used in media and public discussions list column type: String
Relation_To	other individuals (by Individual_ID) that are related to the individual this sample derived from, multiple entries separated by ; list column type: String

Variable	Description
Relation_Degree	relationship degree for relatives mentioned in Related_To, multiple values separated by ; in the same order as Related_To in case of multiple relations. Here, identical refers to identical twins. Samples from the same individual should be encoded through a common Individual_ID. list column type: String allowed values: identical; first; second; thirdToFifth; sixthToTenth; unrelated; other
Relation_Type	relationship type for relatives mentioned in Related_To (e.g. sister_of, child_of, nephew_of), multiple values separated by ; in the same order as Related_To in case of multiple relations list column type: String
Collection_ID	alternative sample identifiers shared by the provider/owner of the sample (e.g. grave 40 skeleton 2), multiple values separated by ; list column type: String
Custodian_Institution	institution that curated the sampled remains at the time of sampling, with name, city and country, multiple entries separated by ; list column type: String
Cultural_Era	the cultural eras approximating the period in which the sampled individual lived (e.g. Danish Bronze Age, Pre-Pottery Neolithic A), if possible taken from an established space-time gazetteer like ChronOntology (https://chronontology.dainst.org) or PeriodO (https://perio.do), multiple entries separated by ; list column type: String
Cultural_Era_URL	urls pointing to definitions of cultural eras, so permalinks complementing the human-readable identifiers in Cultural_Era (e.g. https://n2t.net/ark:/99152/p0zj6g8ks9s, https://chronontology.dainst.org/period/Gx4uxaeTCbbg), multiple entries separated by ; in the same order as Cultural_Era list column type: String

Variable	Description
Archaeological_Culture	the archaeological cultures, technocomplexes, (pottery) styles or political entities the sampled individual can be associated to (e.g. Hallstatt culture (Hungary), Neo-Assyrian Empire), if possible taken from an established space-time gazetteer like ChronOntology (https://chronontology.dainst.org) or PeriodO (https://perio.do), multiple entries separated by ; list column type: String
Archaeological_Culture_URL	urls pointing to definitions of archaeological cultures, so permalinks complementing the human-readable identifiers in Archaeological_Culture (e.g. https://n2t.net/ark:/99152/p0nxc78fxgt, https://chronontology.dainst.org/period/bvLwqFcGyoaL), multiple entries separated by ; in the same order as Archaeological_Culture list column type: String
Country	present-day political country of origin for the sample type: String
Country_ISO	present-day political country expressed in ISO 3166-1 alpha-2 country codes type: String
Location	unspecified location information for the sample, e.g. administrative or topographic region or mountains/rivers/lakes/cities nearby type: String
Site	name of the archaeological site where the sample was found type: String
Latitude	latitude where the sample was found with up to 5 places after the decimal point type: Float allowed range: -90 to 90
Longitude	longitude with up to 5 places after the decimal point type: Float allowed range: -180 to 180

Variable	Description
Date_Type	type of dating information available for the sample, C14 if there is a set of radiocarbon dates in the columns Date_C14_Labnr, Date_C14_Uncal_BP and Date_C14_Uncal_BP_Err whose post-calibration probability distribution is a meaningful prior for the individual's year of death, contextual for any other age information only given in Date_BC_AD_Start, Date_BC_AD_Median and Date_BC_AD_Stop, "modern" for present-day individuals type: String allowed values: C14; contextual; modern
Date_C14_Labnr	lab numbers of C14 ages, multiple values separated by ; in case of multiple dates list column type: String
Date_C14_Uncal_BP	uncalibrated years BP (as in before 1950AD) for the C14 ages as reported by C14 labs, multiple values separated by ; in the same order as Date_C14_Labnr in case of multiple dates, only relevant if Date_Type is C14 list column type: Integer allowed range: 0 to Inf
Date_C14_Uncal_BP_Err	standard deviation (1-sigma ±) for the uncalibrated C14 ages as reported by the C14 labs, multiple values separated by ; in the same order as Date_C14_Labnr in case of multiple dates, only relevant if Date_Type is C14 list column type: Integer allowed range: 0 to Inf
Date_BC_AD_Start	lower (older) bound for the age of the sample in years BC/AD, negative numbers for BC, positive numbers for AD, in case of C14 dates 2-sigma post calibration interval, 2000 for modern samples type: Integer allowed range: -Inf to 2050

Variable	Description
Date_BC_AD_Median	median age of the sample in years BC/AD, for C14-dated samples median, for contextually dated samples simple mid-point of the archaeological intervals, 2000 for modern samples type: Integer allowed range: -Inf to 2050
Date_BC_AD_Stop	upper (more recent) bound for the age of the sample in years BC/AD, counter point to Date_BC_AD_Start type: Integer allowed range: -Inf to 2050
Chromosomal_Anomalies	genetic anomalies on the chromosome level detected for the sample, so extra, missing or irregual portions of chromosomal DNA, including gonosomal and autosomal aneuploidies, multiple entries separated by ; Suggestions include XXY, XYY, XXX, X0, Trisomy21, Trisomy18 list column type: String
MT_Haplogroup	mitochondrial haplogroup derived for the sample. For human data, this should follow the specification on phylotree.org and as reported by the Haplofind or Haplogrep software tools type: String
Y_Haplogroup	Y-chromosome haplogroup derived for the sample. For human data this should follow a syntax with the main branch + the most terminal derived Y-SNP (e.g. R1b-P312) type: String
Source_Material	sampled material, multiple entries separated by ; list column type: String allowed values: petrous; bone; tooth; hair; soft; sediment; other
Nr_Libraries	number of libraries produced for the sample type: Integer
Library_Names	identifiers of the libraries used to generate the genotype data for the sample, multiple values separated by ; list column type: String

.janno file variables *(continued)*

Variable	Description
Capture_Type	capture method for the individual libraries generated for the sample, multiple values separated by ; list column type: String allowed values: Shotgun; 1240K; ArborComplete; ArborPrimePlus; ArborAncestralPlus; TwistAncientDNA; WISC2013; OtherCapture
UDG	udg treatment for the libraries, mixed in case multiple libraries with different UDG treatment were merged type: String allowed values: minus; half; plus; mixed
Library_Built	strandedness of the libraries, "mixed" in case multiple libraries with different protocols were merged type: String allowed values: ds; ss; mixed
Genotype_Ploidy	ploidy of the genotypes for the sample type: String allowed values: diploid; haploid
Data_Preparation_Pipeline_URL	url pointing to a description of the computational pipeline used to generate the genotype data from the source data type: String
Endogenous	fraction (ranging between 0 and 1) of endogenous DNA estimated from SG libraries (before capture) as for example estimated by EAGER, not on target and no quality filter, in case of multiple libraries only the highest values should be reported type: Float allowed range: 0 to 1
Nr_SNPs	number of non-missing SNPs for the sample, counted on the SNP-set stored in the Poseidon package type: Integer
Coverage_on_Target_SNPs	average X-fold coverage across targeted SNP sites after quality filtering type: Float

Variable	Description
Damage	fraction (ranging between 0 and 1) of damage on the 5' end of the libraries used for sequencing, in case of multiple libraries either report multiple values separated by ;, or a single value from the merged read alignment list column type: Float allowed range: 0 to 1
Contamination	(modern) contamination of the sample as measured by the method in Contamination_Meas, multiple values separated by ; (for different methods, in case of multiple libraries report a value from the merged read alignment), the variables Contamination, Contamination_Err and Contamination_Meas must have the same number and order of (non-n/a) entries list column type: String
Contamination_Err	(modern) contamination estimate error of the sample list column type: String
Contamination_Meas	method to measure contamination, should be a software tool (ANGSD, Schmutzi, …) and the respective software versions, details can go to a Contamination_Note column list column type: String
Genetic_Source_Accession_IDs	ENA or SRA accession identifiers pointing to the source data used to generate the genotyping data for the sample, multiple values separated by ;, if multiple are given they should be arranged by descending specificity (e.g. project id > sample id > sequencing run id) list column type: String
Primary_Contact	project lead or first author who generated and published the data for the sample type: String

.janno file variables *(continued)*

Variable	Description
Publication	bibtex keys for the publications where a sample was published (e.g. “AuthorJournalYear“) or “unpublished“, multiple values separated by ;, all must be present with complete BibTeX entries in the Poseidon package’s .bib file list column type: String
Note	arbitrary comments about the sample type: String
Keywords	arbitrary tags, multiple values separated by ; list column type: String

3 .ssf file variables

.ssf file variables

Variable	Description
poseidon_IDs	Poseidon_IDs (in the .janno file) the sequencing entity corresponds to, multiple entries separated by ; list column type: String
udg	udg treatment applied to the library for the sequencing entity type: String allowed values: minus; half; plus
library_built	library preparation method applied for the sequencing entity (single- or double-stranded) type: String allowed values: ds; ss
sample_accession	sample accession code as used in the INSDC databases (including ENA and SRA) to identify the sequencing entity (e.g. SAMEA7050454) type: String
study_accession	study accession code as used in the INSDC databases (e.g. PRJEB39316) type: String

.ssf file variables *(continued)*

Variable	Description
run_accession	run accession code as used in the INSDC databases (e.g. ERR4331996), this should be a unique identifier in a Poseidon package type: String
sample_alias	sample alias defined by the submitter in the raw sequencing data repository type: String
secondary_sample_accession	a secondary sample accession, used in the ENA database for historical reasons (e.g. ERS4811084) type: String
first_public	date (YYYY-MM-DD) the sequencing entity was first made public in the raw sequencing data repository type: Date
last_updated	date (YYYY-MM-DD) the sequencing entity was last updated in the raw sequencing data repository type: Date
instrument_model	name of the instrument used to process the sequencing entity (e.g. Illumina HiSeq 2500) type: String
library_layout	library layout of the sequencing entity (e.g. SINGLE) type: String
library_source	source of the DNA library (e.g. GENOMIC) type: String
instrument_platform	platform, brand or type of the sequencer (e.g. ILLUMINA) type: String
library_name	library identifier, so library name the submitter has entered to the raw sequencing data repository, data entries across which optical duplicates could exist should have matching library names type: String
library_strategy	strategy used to create the library for the sequencing entity (e.g. WGS) type: String

Variable	Description
fastq_ftp	ftp links to the FASTQ files for the sequencing entity in the raw sequencing data repository (e.g. ftp.sra.ebi.ac.uk/vol1/fastq/ERR433/009/ERR4332639/ERR4332639.fastq.gz), multiple entries separated by ; list column type: URL
fastq_aspera	aspera links to the FASTQ files for the sequencing entity in the raw sequencing data repository (e.g. fasp.sra.ebi.ac.uk:/vol1/fastq/ERR433/009/ERR4332639/ERR4332639.fastq.gz), multiple entries separated by ; list column type: URL
fastq_bytes	number of bytes in the FASTQ files, multiple entries separated by ;, must be in the same order as the ftp and/or aspera links list column type: Integer allowed range: 0 to Inf
fastq_md5	md5 hashes of the FASTQ files, multiple entries separated by ;, must be in the same order as the ftp and/or aspera links list column type: String
read_count	number of reads in the sequencing entity type: Integer allowed range: 0 to Inf
submitted_ftp	urls to the originally submitted files before they got converted to FASTQ in the INSDC databases, multiple entries separated by ; list column type: String
submitted_md5	md5 hashes of the originally submitted files before they got converted to FASTQ in the INSDC databases, multiple entries separated by ; list column type: String